

JsDeObsBENCH: Measuring and Benchmarking LLMs for JavaScript Deobfuscation

Guoqiang Chen*
The Ohio State University
Columbus, OH, USA

Xin Jin*
The Ohio State University
Columbus, OH, USA

Zhiqiang Lin
The Ohio State University
Columbus, OH, USA

Abstract

Deobfuscating JavaScript (JS) code poses a significant challenge in web security, particularly as obfuscation techniques are frequently used to conceal malicious activities within scripts. While Large Language Models (LLMs) have recently shown promise in automating the deobfuscation process, transforming detection and mitigation strategies against these obfuscated threats, a systematic benchmark to quantify their effectiveness and limitations has been notably absent. To address this gap, we present JsDeObsBENCH, a dedicated benchmark designed to rigorously evaluate the effectiveness of LLMs in the context of JS deobfuscation. We detail our benchmarking methodology, which includes a wide range of obfuscation techniques ranging from basic variable renaming to sophisticated structure transformations, providing a robust framework for assessing LLM performance in real-world scenarios. Our extensive experimental analysis investigates the proficiency of cutting-edge LLMs, e.g., GPT-4o, Mixtral, Llama, and DeepSeek-Coder, revealing superior performance in code simplification despite challenges in maintaining syntax accuracy and execution reliability compared to baseline methods. We further evaluate the deobfuscation of JS malware to exhibit the potential of LLMs in security scenarios. The findings highlight the utility of LLMs in deobfuscation applications and pinpoint crucial areas for further improvement.

CCS Concepts

• Security and privacy → Web application security; Software security engineering.

Keywords

JavaScript Deobfuscation, Benchmark, Large Language Models

ACM Reference Format:

Guoqiang Chen, Xin Jin, and Zhiqiang Lin. 2025. JsDeObsBENCH: Measuring and Benchmarking LLMs for JavaScript Deobfuscation. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25)*, October 13–17, 2025, Taipei, Taiwan. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3719027.3744871>

1 Introduction

JavaScript (JS) deobfuscation plays a vital role in a range of web analysis and security tasks, including vulnerability detection [47, 71], malware analysis [34], and program comprehension [73]. At

its core, deobfuscation seeks to reverse the transformations applied during JS obfuscation, thereby rendering the code more readable and amenable to analysis. Empirical studies show that over 40% of JS code found on online platforms is obfuscated or minified [61]. This practice is also widespread in mobile applications, particularly within miniapp ecosystems, where most apps (benign or malicious) employ JS obfuscation [80, 81]. Notably, such obfuscation has been shown to increase the false negative rate of malicious JS detectors by 21.8% [61], posing significant challenges for malware analysis and threat mitigation.

Unfortunately, JS deobfuscation presents significant challenges. Real-world JS programs often undergo sophisticated transformations and the combination of several transformations is common [61, 72], which can substantially alter both the literal content and the structure of the program. Deobfuscation involves modifying the syntax while maintaining the semantics of the code, aiming not only to reverse obfuscations but also to simplify the code and enhance its readability [48, 60]. Evaluating the correctness, simplification, and readability of deobfuscated code is particularly challenging due to the absence of reliable ground truth and effective evaluation metrics.

Recent advances in Large Language Models (LLMs), such as ChatGPT, have marked a significant milestone in the field of code deobfuscation [75]. These models demonstrate a powerful ability to detect complex patterns and generate human-like text, providing security analysts with new, AI-powered tools to better understand obfuscated code. More specifically, these tools allow security analysts to interact directly with the language model, submitting obfuscated code snippets and requesting insights or clearer versions. This conversational approach significantly streamlines the deobfuscation process, reducing both the time and effort traditionally required for manual analysis and thereby enhancing the efficiency and effectiveness of threat detection. However, despite these achievements, the potential of LLMs for deobfuscating JS code specifically has yet to be fully evaluated, primarily due to the absence of a suitable benchmark to measure their capabilities in this specific context.

To bridge this gap, we present JsDeObsBENCH, the first JS deobfuscation benchmark tailored for assessing JS deobfuscation with LLMs. This benchmark encompasses a large-scale and execution-verifiable dataset, containing 36,260 unique obfuscated JS programs with ground truth and 4,515 malicious obfuscated JS programs. The dataset is crafted by applying seven common obfuscation transformations to a diverse set of JS programs, which are subsequently subjected to data sanitization to ensure that the programs are (1) executable with test cases, (2) with appropriate lengths, and (3) discarding those with duplicate functionalities. Additionally, JsDeObsBENCH incorporates an automated evaluation pipeline equipped with four comprehensive evaluators. These systematic evaluators can not only evaluate the deobfuscation effectiveness by examining

*Both authors contributed equally to this work.



This work is licensed under a Creative Commons Attribution 4.0 International License. CCS '25, Taipei, Taiwan

© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1525-9/2025/10
<https://doi.org/10.1145/3719027.3744871>

both the syntactical and semantic correctness, but also assess the degree of simplification in the deobfuscated code and its similarity to the original JS programs, serving as an indicator of code readability.

With JsDeObsBENCH, we have evaluated six newly released state-of-the-art LLMs from four model families: CodeLlama [66], Llama-3.1 [51], Codestral [52], Mixtral [42], Deepseek-Coder [36], and a close-source model GPT-4o [54], which are top-ranked on the EvalPlus leaderboard [5] based on their advanced coding capabilities. To optimize their deobfuscation performance, we utilize the in-context learning capacities of LLMs, crafting specific prompts that guide the models in our deobfuscation task through demonstrative examples. Furthermore, we have selected two existing deobfuscators as baselines, jointly considering their comprehensive functionalities, popularity, and scalability.

Our evaluations have drawn a set of results and findings. First, among the LLMs, the leading GPT-4o model demonstrates the best overall performance, and Codestral outperforms the others. JS programs subjected to string obfuscation and code compact transformations are the most challenging and the easiest, respectively, for LLMs to deobfuscate. Second, many LLM-deobfuscated JS programs struggle with syntax and execution evaluations, with average failure rates of 2.76% and 37.40%, respectively, while our baseline methods rarely encounter such failures. Both LLMs and our baselines are sensitive to the number of obfuscation transformations applied. Third, LLMs demonstrate superior performance in simplifying obfuscated JS code compared to our baselines (e.g., achieving a 1.72× better code simplification score). LLM-deobfuscated code also yields better CodeBLEU with the original code than the baselines, indicating a more readable deobfuscation. Finally, the LLMs also exhibit a potential in deobfuscating JS malware, and especially perform well in simplifying the obfuscated malicious programs. Moreover, we carried out more investigations on in-context learning, code length, and case studies, which are presented in the Appendix §A for interested readers.

Contributions. We make the following contributions:

- We construct a large-scale, execution-verifiable dataset for deobfuscation assessment by applying prevalent obfuscation transformations to quality-sanitized JS programs.
- We introduce a novel LLM deobfuscation benchmark, JsDeObsBENCH, with an automated and comprehensive deobfuscation evaluation pipeline, designed to assess both syntactical and semantic correctness, as well as the effectiveness of code simplification and readability.
- We conduct systematic experiments on six advanced LLMs, yielding a set of findings, which shed light on their practical applications in deobfuscating JS code and outline potential future advancements in LLM deobfuscation design.

Artifact and Leaderboard. We have released our datasets and code in <https://github.com/OSUSecLab/JSDeObsBench>, and created a leaderboard in <https://jsdeobf.github.io/>.

2 Background and Related Works

In this section, we discuss the background and related works by first introducing our problem definitions in §2.1 and then presenting related works in §2.2.

2.1 Problem Definition

Given a JavaScript (JS) program P and a set of obfuscation transformations T , the obfuscator O applies a configuration series of transformations $S = t_1, t_2, \dots, t_n$ (where $t_i \in T$) to produce an obfuscated version of the program, denoted as P' : $P' = O_S(P)$. The JS deobfuscator D then reverses the transformations applied to P' , resulting in a form P'' : $P'' = D(P')$, such that P'' is sufficiently similar to P , or at least equivalently comprehensible.

In this paper, we explore the application of LLMs as deobfuscators (D) and evaluate their effectiveness against existing obfuscation techniques, including both minification and advanced transformations (e.g., control flow flattening). Minification [72] specifically refers to the process of reducing code size by removing useless code elements, while advanced transformations are designed to produce more complex and sophisticated programs.

2.2 Related Works

JavaScript Obfuscation and Detection. As the web becomes increasingly ubiquitous, obfuscation is extensively employed in JS code to conceal the intentional behavior of scripts. Obfuscation can be categorized into several types based on the targeted JS program components, such as names, data, control flows, and layouts [49]. Open source tools such as JavaScript-Obfuscator [6] integrate many obfuscation transformations commonly used in practice. Additionally, the web community has proposed advanced obfuscation approaches to enhance the effectiveness and optimize for specific domains [49, 63, 83].

Since obfuscation is prevalent in malicious websites, JS obfuscation detection remains an active research area. Several approaches use static information, e.g., syntactical data, derived from JS files to pinpoint obfuscation features [29, 65, 68]. Other research employs machine learning-based methods to identify obfuscated JS code [35, 46, 72]. Although obfuscation detection results are valuable, they do not fully address the challenges posed by obfuscation, and still require significant human efforts to analyze the obfuscated code.

JavaScript Deobfuscation. While recognizing the significance of deobfuscation, only a modest number of studies and toolkits have been introduced to counteract JS obfuscation. Within this domain, a majority of these research efforts adopt learning-based approaches for recovering variable names [48, 60, 74, 76]. However, these works primarily focus on reversing transformations that target name obfuscation only. Meanwhile, a comprehensive set of deobfuscators has been developed by the open-source community to broadly address the challenges of obfuscation [3, 6, 9, 10]. For example, Synchrony [9] is a popular tool that removes common obfuscation transformations in JS. Although these open-source deobfuscators cover a wider range of obfuscation transformations, we observe that their heuristic, rule-based approaches are ad-hoc, lacking robustness and generalizability, and often result in less readable code compared to learning-based methods.

Large Language Models. Recently, LLMs have achieved significant success in various real-world security applications [27, 44, 45, 69, 70]. These models excel at capturing context-sensitive representations of natural languages [58]. As an early and representative example, the encoder-only language model, BERT [31], utilizes

masked language modeling to learn bidirectional word representations with a transformer encoder. To overcome challenges about the generalizability of BERT, the decoder-only LLMs, including the well-known ChatGPT model, are designed to handle real-world tasks requiring robust in-context learning abilities [78]. A key feature of these advanced LLMs is their in-context learning capacity, *i.e.*, the ability to acquire task-specific information through natural language instructions and demonstration examples without the need for training or fine-tuning on specific task data [24]. These instructions and examples are usually provided via natural language descriptions, often known as prompts [50].

3 Overview

In this section, we first outline the challenges encountered in this research in §3.1. Then, we detail our insights and proposed solutions in §3.2. Finally, we introduce the JsDEOBSBENCH workflow in §3.3.

3.1 Challenges

Lack of Obfuscated JS Dataset with Ground Truth. The absence of a diverse and verifiable obfuscated dataset with ground truth presents a significant challenge. For JS deobfuscation, most existing studies [53, 61, 72] rely on obfuscated programs from online websites and public malware samples. However, such obfuscated programs are closed-source, preventing access to the ground truth, which is essential for statistical evaluations and necessary for benchmarking. Additionally, real-world obfuscated JS programs could undergo various obfuscation transformations, complicating the reverse engineering of transformation configurations, and consequently very challenging to reconstruct the ground truth data.

Constraints on LLM Interpretability and Knowledge Gap. Employing LLMs to deobfuscate JS programs introduces challenges of mitigating knowledge gaps and ensuring evaluation fairness. Most state-of-the-art LLMs are trained predominantly on natural language corpus and have limited exposure to obfuscated code, which creates a knowledge gap regarding the selection of effective LLMs for deobfuscation tasks. This gap is exacerbated by the poor interpretability of LLMs, complicating efforts to prevent data leakage. Specifically, the tendency of LLMs to potentially memorize [25] training samples prevents us from using publicly accessible obfuscated JS programs.

Missing Comprehensive Evaluation Metrics to Assess Deobfuscation Performance. The comprehensive evaluation metrics for assessing the correctness and effectiveness of deobfuscation are missing. Deobfuscation, aiming to revert obfuscation transformations, significantly alters the syntax of programs and their semantics in case of errors. Therefore, evaluation metrics are required to assess both the correctness of syntax and code semantics of deobfuscation results. Additionally, one of the key purposes of deobfuscation is to facilitate human analysts' comprehension of JS programs. Therefore, the readability and understandability of deobfuscated code must be included in this evaluation process. However, current metrics [23, 48, 60] for JS deobfuscation evaluation predominantly focus on assessing the correctness of variable name recovery, which fails to meet the needs for our evaluation framework.

3.2 Insights and Solutions

A Large-scale and Execution-verifiable Deobfuscation Assessment Dataset. In this paper, we develop a large-scale obfuscated JS dataset from scratch to benchmark deobfuscation performance. Observing the difficulties in obtaining ground truth for existing resources (*e.g.*, online websites), we note that real-world obfuscated JS frequently undergoes typical obfuscation transformations. Moreover, since our evaluations necessitate assessing code semantics, this characteristic can be evaluated through coding challenges, *i.e.*, verifying program outputs based on given inputs. Additionally, we discover that LLM pretraining benefits from datasets curated from online resources. These observations prompt us to construct a obfuscated JS dataset from scratch using our customized obfuscation configurations, which also helps minimize its leakage to models.

Code-specific LLM Selection and In-context Learning. Recently, we have witnessed the emergence of numerous LLMs designed for domain-specific tasks. Although none is tailored for JS deobfuscation, the code-specific LLMs have shown outstanding abilities in code comprehension, the fundamental capability for our task. Consequently, we have selected open-source LLMs that excel in code comprehension as evidenced by their rankings on the EvalPlus leaderboard [28]. Additionally, to optimize their deobfuscation performance, we employ in-context learning [24] with carefully designed prompts. In this process, we instruct LLMs to perform JS deobfuscation by providing both task description and demonstration examples within the query context.

Deobfuscation Assessment using Comprehensive Evaluators. To conduct thorough evaluations of deobfuscation outputs, we introduce a chain of four evaluators that assess (1) syntax correctness, (2) execution correctness, (3) complexity reduction, and (4) code readability. Specifically, we first verify the syntactical correctness of the deobfuscation outputs, rather than relying on existing metrics. For the syntactically correct outputs, we then evaluate their code semantics by validating executing outputs given the input we collected. For measuring the effectiveness of complexity reduction, we employ a Halstead-length [37] based JS code simplification evaluation metric. Finally, to determine how well deobfuscators produce easy-to-read code, we assess code similarity between the original programs and the deobfuscation outputs, which serves as an indicator of the code's readability and understandability.

3.3 JsDEOBSBENCH Workflow

Figure 1 provides an overview of the JsDEOBSBENCH workflow. At a high level, it consists of three key steps: (i) building evaluation dataset by obfuscating diverse and behavior-aware JS programs using the common obfuscation transformations, (ii) creating an in-context learning pipeline to query the LLMs under our testing for JS deobfuscation, and (iii) assessing the LLM deobfuscation outputs using our comprehensive evaluators.

4 Detailed Design and Implementation

4.1 Dataset Construction

Existing methodologies collect obfuscated JS programs as datasets but fail to address the challenges we previously outlined in §3.1. In this work, we carefully design strategies and pipelines to construct

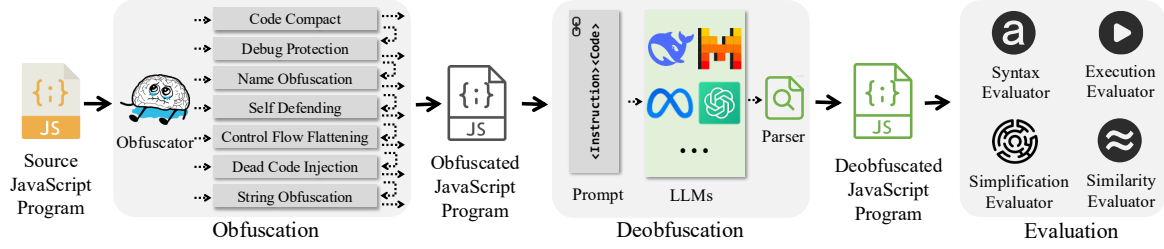


Figure 1: JsDeObsBENCH Overview.

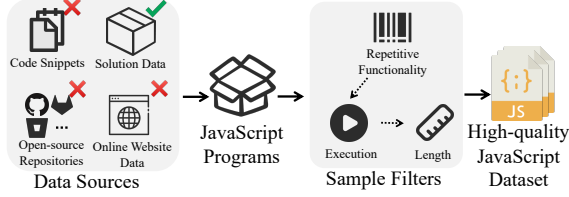


Figure 2: JS Source Selection and Filtering. Among the four data sources, we select the solution data by comprehensive consideration.

the obfuscated dataset from scratch, which requires curating high-quality and functionally diverse JS programs. Figure 2 presents our data curation process for the obfuscated dataset, which includes data source identification and JS sample filtering:

Data Source Identification. By exploring online JS programs, we first identify four potential sources of JS programs: (1) online code snippets (e.g., those from StackOverflow), (2) open-source repositories, (3) online website scripts without obfuscation, and (4) JS solutions for coding challenges. To build a large-scale and diverse benchmarking dataset, we have established three criteria for selecting reliable sources:

- **Complete Human-created Programs.** The source JS programs should be complete and written by human developers for solving practical, real-world problems. This criterion ensures that the test dataset mirrors the complexity typically encountered in real-world environments, providing a robust foundation for evaluating LLMs’ deobfuscation capabilities in realistic conditions.
- **Diverse Functionalities and Styles.** We expect the data source to exhibit a high degree of diversity. Primarily, this diversity should manifest functionally, encompassing solutions to various problems. Additionally, it should reflect the difference in developers’ programming styles.
- **Execution Verifiable.** Given that deobfuscation should not alter the functional semantics of the code, any semantic changes should be detectable. Inspired by the HumanEval benchmark [28], which is widely adopted in assessing code-LLMs (e.g., ChatGPT and CodeLlama), the program test cases are imperative, typically comprising a set of inputs and their corresponding outputs. Therefore, we expect the source programs to be execution-verifiable with test cases.

Based on these criteria, we observe that online code snippets often consist of incomplete or simplistic scripts primarily used for

demonstrations, instruction, or testing, limiting their functionality. Open-source repositories, while diverse in functionalities and programming styles, present challenges in building and execution due to the need for project-specific configurations and environments, making scalability and verification difficult. Moreover, many scripts from online websites are also already obfuscated [72], and accurately distinguishing between obfuscated and non-obfuscated JS programs remains a challenge [61].

In this paper, we select the JS solutions for coding challenges as our data source. Specifically, we have opted to use the programs from CodeNet [57], which have been widely used for code-specific evaluation tasks [32, 56]. Note that the selection of coding-challenge solutions is a common practice in LLM evaluation benchmarks, such as HumanEval [24] and EvalPlus [5]. CodeNet contains 13.9 million code samples across 4,053 programming problems in 55 languages, derived from the code submitted to two online judge websites (i.e., AIZU [77] and AtCoder [30]). These samples come from different users and are programmed for a variety of problems with test cases for correctness validation. From the CodeNet dataset, we unpack and extract all 58,395 JS programs as our initial dataset, which covers solutions for 1,935 distinct problems, showing a diverse range of development purposes.

JS Sample Filtering. While CodeNet offers a high-quality set of JS programs, several concerns still persist: (1) CodeNet contains samples that do not pass online judgment. (2) Some samples exceed 25k characters beyond the maximum output length of typical LLMs, potentially hindering effective evaluation. (3) The code samples often exhibit repetitive functionality, which could lead to unnecessary evaluation overhead. To address these problems, as shown in Figure 2, we have developed three filters to clean up our initial dataset:

- **Execution Filter.** To guarantee valid test programs, this filter assesses each JS program against its corresponding test cases to verify their execution correctness. Only those samples that pass all their respective tests are retained.
- **Length Filter.** We have employed the filter with minimal and maximum length boundaries that align with the common context window sizes of our target LLMs, ensuring that the samples included in our dataset are feasible and efficient for LLM inference.
- **Repetitive Functionality Filter.** In the initial dataset, there can be multiple JS programs solving the same problems. Therefore, this filter is designed to retain a single sample for each distinct

problem to avoid redundant evaluation, which yields a dataset with constant functional diversity.

Table 1: JS Source Dataset Statistics with Sample Filtering. The initial 58,395 samples are sequentially reduced to 1,298.

Filter	# Programs	# CodeNet Problems
N/A (Initial)	58,395	1,935
Execution	24,571	1,613
Length	18,443	1,298
Repetitive Functionality	1,298	1,298

To obtain a high-quality JS dataset, we have sequentially applied our three filters on our initial 58,395 CodeNet JS programs. Table 1 summarizes the statistics of our data filtering process, showcasing the reduction in the number of remaining samples and CodeNet problems as we apply each filter. Our final dataset includes 1,298 JS code samples, each tackling a unique programming problem. These samples have been rigorously vetted to ensure diverse functionality, execution correctness, and moderate length. This curated dataset forms the foundation for our obfuscated dataset construction, *i.e.*, serving as the input source JS programs as shown in Figure 1.

JS Obfuscation. We aim to construct an obfuscated JS dataset to evaluate the performance of LLMs in deobfuscating code. Having acquired the high-quality source JS programs, we apply obfuscation transformations to create this dataset. To accurately replicate real-world obfuscated programs produced by developers, we initially investigated transformation techniques supported by popular obfuscators [8, 12, 15–17], and reviewed existing empirical studies [53, 61, 72]. Finally, we have identified the following prevalent obfuscation transformations commonly adopted in practice:

- **Code Compact** reduces the program size by removing unnecessary whitespace, newlines, and comments, typically yielding code in one line.
- **Name Obfuscation** alters the identifiers of variables and functions into meaningless patterns to obscure their semantics, such as hexadecimal formats (e.g., `_0xabc123`) and mangled short names (e.g., `a`, `b`, `c`).
- **String Obfuscation** removes string literals from the original code to hinder static analysis and dynamically restores obscured strings to their original form and location in the code at runtime.
- **Dead Code Injection** randomly adds useless dead code to code nodes to significantly increase the code size.
- **Control Flow Flattening** rebuilds the control flow of the program into a single-level structure and generally uses switch statements to separate the original basic blocks.
- **Debug Protection** inserts code that detects and disrupts debugging tools to terminate execution or alter the behavior when a debugger is detected.
- **Self-defending** incorporates checksums or self-modifying codes to prevent attempts at formatting and identifier renaming.

To construct this dataset, we have evaluated existing JS obfuscators, considering the popular ones from both the industry [8, 11] and the open-source community [15–17]. In particular, we select the

most popular obfuscator, JavaScript-Obfuscator [17], which has over 13.1k stars on GitHub and is actively maintained. In addition to its popularity among developers, it is widely used by prior research [53, 61, 72], indicating a broad consensus on its popularity and representativeness. JavaScript-Obfuscator has also shown superior performance in comparative evaluations of JS obfuscators [59]. Importantly, it offers flexible configuration options, allowing us to effectively combine multiple obfuscation transformations to meet our specific needs.

Transformation Combinations. In practice, multiple obfuscation transformations are often used together to obfuscate the JS programs [72]. Consequently, we not only apply individual transformations but also combine them to create a complex obfuscated JS dataset, as shown in Figure 1. JavaScript-Obfuscator supports the simultaneous application of multiple transformations, which are applied to the input program in a predetermined order that users do not need to concern themselves with. Therefore, when selecting a certain number of transformations, we perform multiple samplings, obtaining a possible combination of transformations each time. In our evaluation, we randomly selected 20% of all 127 ($C_7^1 + C_7^2 + \dots + C_7^7$) possible combinations, as evaluating LLMs for all of them results in excessive overhead. For an effective assessment, we ensure that combinations with different numbers of transformations can be sampled uniformly. The average scores of combinations with a certain number of transformations are calculated as the final results. Eventually, we construct an obfuscated JS dataset with 36,260 unique programs using JavaScript-Obfuscator.

JS Malware Dataset. To further investigate how well LLMs can de-obfuscate malicious JS programs, we construct a dataset with 4,515 obfuscated malicious JS programs following the same aforementioned approaches. More specifically, we first collect malicious JS programs from Hynek Petrak [1] and GeeksOnSecurity [20], which contain 40,830 real-world JS malware in total. Next, our three filters are applied to these samples to sanitize the raw dataset to guarantee an effective evaluation dataset. However, there is a challenge in collecting ground truth, *i.e.*, these samples have no available test cases to check the execution semantics, and thus their semantic correctness cannot be directly evaluated by program inputs and desired outputs. By instrumenting the code, we observe that malicious JS code expresses semantics via program behavior, including modifying registries, requesting URLs, writing files, etc. Therefore, we address this challenge by checking program behavior, following existing malware detection research [41, 67]. Specifically, we record the behavior trace of a JS malware with Box-js [22], which is a tool designed for JS malware execution, instrumentation, and behavior tracking. In addition to our proposed filters, we apply a malware-specific filter by considering a sample to be malicious only if its malicious behaviors are recorded by Box-js. After performing obfuscation transformations on the filtered samples, we run the obfuscated samples again to check if they have the same behavior as the original malicious samples.

4.2 Deobfuscation by LLMs

In this paper, we aim to evaluate how state-of-the-art LLMs can de-obfuscate JS programs. To obtain the optimal performance of LLMs, we instruct the LLMs on our deobfuscation task, using specifically

Table 2: LLMs Selected in JSDeObsBENCH Evaluations.

Model	Size	Model ID	Context Length
CodeLlama	7B	CodeLlama-7b-Instruct-hf	16k
Llama-3.1	8B	Llama-3.1-8B-Instruct	128k
Codestral	22B	Codestral-22B-v0.1	32k
Mixtral	7B	Mixtral-7B-Instruct-v0.3	32k
Deepseek-Coder	7B	DeepSeek-Coder-V2-Lite-Instruct	128k
GPT-4o	N/A	GPT-4o-2024-08-06	128k

designed prompts based on in-context learning [24, 43]. Also, as stated in §3.2, we select top code-LLMs for the deobfuscation.

In-context Learning for Deobfuscation. The advanced LLMs possess a robust capability of instruction following [82], meaning that they can comprehend natural language instructions, or prompts, and respond appropriately. To tailor LLMs for JS deobfuscation, we have designed in-context learning instructions, namely zero-shot and few-shot prompts. Initially, we created a zero-shot prompt (detailed in Figure 9a in Appendix), an instruction without any demonstration examples, inline with established practices in prompt engineering [50]. In this prompt, we provide both the task description and the obfuscated JS code, enclosed within three backticks to delineate the code format. Additionally, we append a short instruction for the expected output of deobfuscated code. We observe that this directive can effectively guide LLMs to produce clear, well-formed outputs devoid of noisy text.

In addition to the zero-shot prompt, we have also developed a one-shot prompt (as depicted in Figure 9b in Appendix). This design choice is motivated by evidence suggesting that including demonstration examples can enhance domain-adaptation performance [24], a finding corroborated by our evaluation results (see Appendix §A.1). While more prompt engineering techniques exist, such as chain-of-thought [79], it has been found that these methods do not provide substantial performance improvements sufficient to justify their significantly higher computational costs [44]. Considering real-world JS deobfuscation tasks usually involve analyzing numerous input programs [72], we focus on using the one-shot prompt to jointly optimize effectiveness and efficiency.

LLM Selection. In this paper, we select state-of-the-art LLMs based on criteria including transparency, reproducibility, accessibility, and code-specific performance. First, we give preference to models from the open-source community over commercial ones to ensure transparency and reproducibility in our assessments. Open-source LLMs provide full control over inference configurations, thus avoiding the opaque settings of black-box systems that can affect the accuracy of evaluations. These models also are more viable and controllable for scalable tasks like our JS deobfuscation with millions of requests. Although no LLMs are specifically designed for JS deobfuscation, we focus on those tailored for programming tasks, as deobfuscation poses unique challenges akin to complex program analysis. That is, we prioritize code-LLMs over general-purpose ones due to their enhanced code comprehension capabilities, which are more likely to provide an effective foundation for deobfuscation. For example, CodeLlama has been fine-tuned on JS programs, making it a suitable choice for our needs [66]. Even with the preference towards open-source code-LLMs, we still introduce a leading model in the

general domain, GPT-4o, to provide additional references. Eventually, we have selected six individual models from four different model families, which rank top on the EvalPlus leaderboard [5] (a rigorous and esteemed evaluation framework for code-LLMs). Table 2 presents the details of these six models.

4.3 Deobfuscation Evaluators

The assessment of deobfuscation outputs poses a significant challenge due to the limitations of existing evaluation metrics [48, 60, 74, 76], which cannot provide comprehensive results. As stated in §3.2, we have proposed four evaluators designed to jointly assess the deobfuscation performance of LLMs. These include a syntax evaluator, an execution evaluator, a decomplexity evaluator, and a similarity evaluator. Each evaluator targets specific aspects of the deobfuscation process to ensure a thorough evaluation of LLMs. In the following, we provide details of our four evaluators.

(1) Syntax Evaluator. The deobfuscation outputs generated by LLMs possess randomness and openness, and it is uncertain whether they follow the correct syntax specification. To evaluate this correctness, we have built a syntax evaluator based on *esprima* [39], which is a standard-compliant JS parser with full support for ECMAScript 2017 [33]. We consider a piece of deobfuscated code to be syntactically correct only if the parser correctly parses it.

(2) Execution Evaluator. To further verify the semantic correctness of the deobfuscated code, we have designed an automatic execution evaluator. This evaluator utilizes the test cases accompanying with each JS program to validate the consistency of the functionality. Considering the security risks associated with executing unsanitized code, we have established an isolated environment using Docker to safely execute the JS code samples. Within this environment, we utilize *Node.js* [14] to run the JS programs with the corresponding inputs. A program is only deemed to have successfully passed the execution check if it clears all designated test cases. Especially, for the malicious JS programs, we compare their behavior traces before and after deobfuscation for the execution evaluation.

(3) Simplification Evaluator. JS obfuscation typically increases the complexity of programs to hinder the analysis. Deobfuscation aims to reduce this complexity, making the code easier to understand and analyze. To assess the effectiveness of deobfuscation in simplifying code, we propose a simplification evaluator that assesses the reduction in complexity of the deobfuscated code compared to obfuscated code.

Drawing inspiration from previous research [38, 40], we gauge code complexity with halstead length [37] of code (*HLoC*), which is calculated from the numbers of operators and operands. A reduction in halstead length is intuitively considered a decrease in the difficulty of understanding the code. To quantify this effect, we introduce the simplification score to measure the degree of complexity reduction achieved by deobfuscation.

$$S = \frac{HLoC_{obf} - HLoC_{deobf}}{HLoC_{obf}} = 1 - \frac{HLoC_{deobf}}{HLoC_{obf}} \quad (1)$$

(4) Similarity Evaluator. As discussed in §2.2, deobfuscation should have to enhance program readability and help developers

better understand the programs. Ideally, measuring the readability of the deobfuscated code would involve human assessment, but this method requires significant manual effort and lacks scalability. To automate this process, we propose to calculate the code similarity between the original and the deobfuscated JS programs, *i.e.*, P and P'' defined in §2.1, respectively. We assume high-quality deobfuscated code (P'') should maintain readability and closely resemble the original code (P). Therefore, we measure the literal and structural similarities between P and P'' by jointly comparing their abstract syntax trees (ASTs) and data flow graphs (DFGs) using CodeBLEU [62], which evaluates syntactic alignment and data structure fidelity.

In JsDEOBSBENCH, each deobfuscated JS program undergoes a sequential assessment by our four designated evaluators. Initially, the syntax evaluator is employed to verify the syntax correctness of the program. Only the programs that pass this initial syntax check are then subjected to the execution evaluator. If the deobfuscated code successfully passes the execution evaluation, it is deemed an effective and successful deobfuscation. For deobfuscated programs with correct syntax, we proceed to further assessment using our simplification and similarity evaluators.

5 Evaluation

We have implemented JsDEOBSBENCH in Python and leveraged open source projects including PyTorch [18], transformers [21], DeepSpeed [2], Docker [4], JavaScript-Obfuscator [17], Node.js [14], and esprima [39]. Our evaluation environment is configured on a Ubuntu 22.04 server with an AMD EPYC 7513 32-core CPU, 1TB RAM, 1TB storage disk, and eight NVIDIA A100 GPUs with 80 GB VRAM each.

Our evaluations aim to answer the following questions:

- **RQ1:** What is the overall performance of LLMs in deobfuscating JS programs?
- **RQ2:** Can LLMs and our baselines produce deobfuscated code with correct syntax and semantics (*i.e.*, passing syntax and execution checks)?
- **RQ3:** How readable are LLMs-deobfuscated programs (*i.e.*, complexity reduction and similarity to the original code)?
- **RQ4:** Can LLMs deobfuscate malicious JS programs?

5.1 Experiment Setup

Baselines. For a deeper understanding of LLMs' deobfuscation performance, we have compared them against existing JS deobfuscators. In selecting representative deobfuscators, we consider factors including popularity among developers, comprehensive functionalities, efficiency, and accessibility. For this, we have evaluated existing deobfuscation tools from the open-source community, including JS-deobfuscator [6], Synchrony [9], De4js [10], JSNice [19], jsNaughty [13], obfuscator-io-deobfuscator [3], and DeMinify [48].

Among these, JSNice and De4js offer web-based interfaces, requiring manual input and extraction of deobfuscation results, posing scalability and efficiency challenges. The other tools are hosted on GitHub, with JS-deobfuscator and Synchrony being the most popular, garnering 827 and 939 stars at the time of this

Table 3: Statistics on Average Input Length (at the Character Level) of LLM Input across Obfuscation Transformations.

Transformation	Obfuscated Code	w/ Zero-shot Prompt	w/ One-shot Prompt
Code Compact	468.23	850.23	1,531.23
Debug Protection	2,390.95	2,772.95	5,319.95
Name Obfuscation	834.22	1,216.22	2,074.22
Self Defending	1,255.52	1,637.52	3,140.52
Control Flow Flattening	1,255.52	1,637.52	3,140.52
Deadcode Injection	1,426.77	1,808.77	3,169.77
String Obfuscation	2,434.51	2,816.51	5,240.51

writing, respectively. In contrast, DeMinify has only one star, indicating the lesser popularity. The popularity of JS-deobfuscator and Synchrony is attributed to their comprehensive functionalities—they are designed as general-purpose deobfuscators capable of reversing common obfuscation transformations. In contrast, JSNice and DeMinify focus primarily on recovering variable names. Based on these observations, we have selected JS-deobfuscator and Synchrony for our study, as they offer a balance of comprehensive functionalities, popularity, and scalability, making them suitable baselines for our evaluations.

Inference Settings. We download publicly available model weights for LLMs listed in Table 2 from Huggingface [7]. For GPT-4o, we directly use the API [55] provided by OpenAI. The models are running locally in half-precision of FP16 for an efficient inference. We maximize the input length by setting the batch size to 1 to support the inference of obfuscated samples as long as possible, as a larger batch size draws more GPU memory. Table 3 presents the statistics of average input length across obfuscation transformations, in which we observe that string obfuscation and code compact transformations produce the longest and shortest obfuscated code, respectively. The demonstration example in one-shot prompts can increase the input length by 83.01% compared to zero-shot prompts. For deobfuscation output, we set the maximum generation length to 2,048 tokens, which can accommodate the longest original JS program we collected. In other words, ideal inference would output the full code without worrying about output truncation. To maintain consistency in our evaluations, We set both top_p and top_n to 1 to obtain the maximum probability prediction, and we also keep the temperature to 0.1 consistently.

5.2 RQ1: Overall Effectiveness of LLMs

Table 4 displays the overall performance across LLMs averaged in obfuscation transformations. For each metric, we highlight the best performance across LLMs and baselines, respectively. For syntax and execution evaluations, we provide the ratio of successfully passing the check among the JS programs tested. The simplification scores are calculated by the reduction of halstead length after the deobfuscation, as shown in Equation 1. In the similarity evaluation, we report the CodeBLEU score which comprehensively encompasses textual similarity, structural similarity, and data flow between deobfuscated code and original code. Meanwhile, we also report the average time overhead in seconds for deobfuscating JS programs.

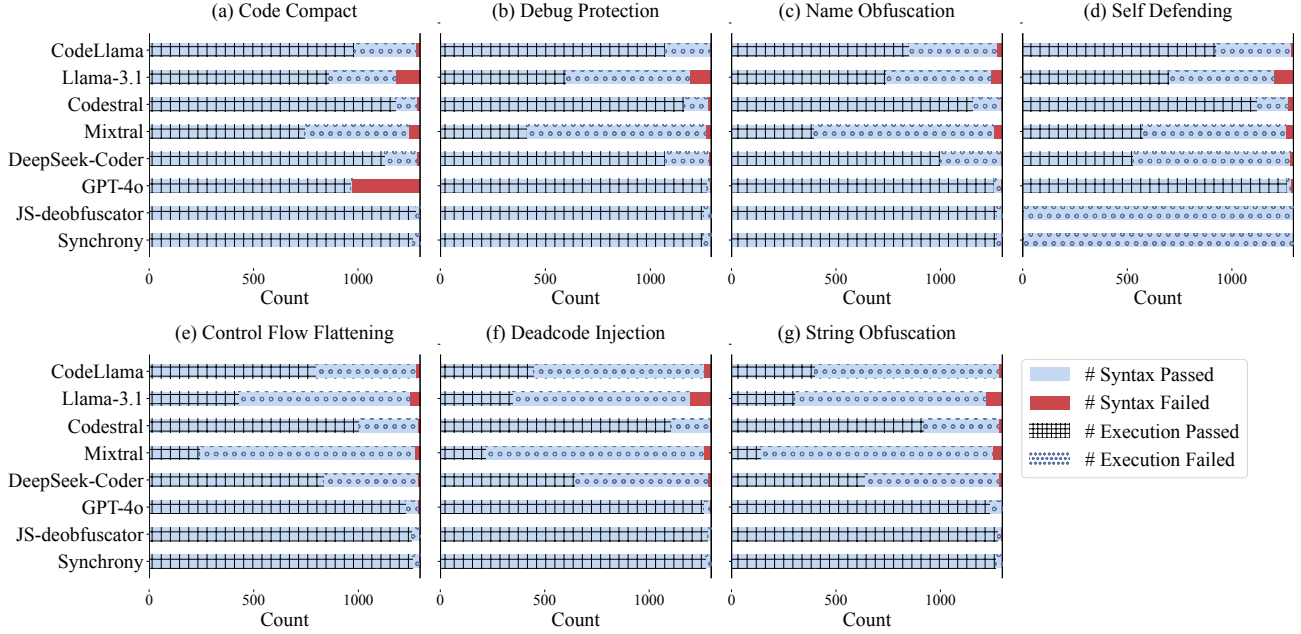


Figure 3: Syntax and Execution Correctness of LLMs and Baselines across Obfuscation Transformations.

Table 4: Overall Deobfuscation Performance of LLMs and Baselines.

Model	Syntax Correctness	Execution Correctness	Simplification Scores	Similarity Scores	Time(s) Overhead
CodeLlama	0.9865	0.6009	0.3361	0.4999	3.77
Llama-3.1	0.9352	0.4373	0.1974	0.4923	2.86
Codestral	0.9900	0.8416	0.3596	0.6096	9.62
Mixtral	0.9721	0.2995	0.2452	0.4481	2.71
Deepseek-Coder	0.9906	0.6425	0.2671	0.5527	3.15
GPT-4o	0.9599	0.9342	0.3825	0.6702	N/A
JS-deobfuscator	1.0000	0.8396	0.0199	0.4936	0.27
Synchrony	1.0000	0.8354	0.2214	0.5941	0.18

Overall, we observe that the GPT-4o model performs the best in terms of execution correctness (0.9342), code simplification (0.3825), and similarity score (0.6702), but it is slightly underperformed by Deepseek-Coder (0.9906) in syntactic correctness. Surprisingly, the best results from LLMs lead our baseline approach almost across the board, especially achieving a 16.11% lead in the simplification score. And both GPT-4o and Codestral even successfully deobfuscated more JS programs than the baselines, showing an impressive accuracy in capturing and recovering the semantics of the obfuscated code. On average, LLMs are able to generate syntactically correct code in 97.23% of predictions and guarantee semantically correct execution with 60.93% probability. Compared to 100% and 83% achieved by the baselines, however, generating syntactically and semantically correct code is still a problem that LLMs must be

improved to overcome. On the other hand, for syntactically correct deobfuscation generation, LLMs perform well in terms of code simplification and readability. Specifically, LLMs achieved simplification scores and similarity scores that are on average 12.13% and 1.6% higher than baselines. Another disadvantage of LLMs is the time taken to perform deobfuscation, the average reasoning speed of LLMs lags substantially behind baselines, with a gap of about 20×.

Making comparison between LLMs, the results show that the code-specific models generally outperform the general-purpose models within a LLM family which usually share the similar model architecture and training process. For instance, the CodeLlama has 5.13% higher score than the Llama-3.1 on syntax correctness, and the Codestral achieved 54.21% better performance than the Mixtral on execution correctness. Interestingly, beyond the same model family, code-LLMs are also superior than the general LLMs at generating syntactically correct deobfuscated JS code, even when compared to the leading GPT-4o, *i.e.*, 95.99% *v.s.* 99.06% by Deepseek-Coder. The better syntactic sensitivity could be due to more training on code dataset and domain tasks. Creating an expert model on JS deobfuscation based on a code model is a better option.

In addition to the overall performance, we further present the detailed scores on transformation-specific for reference, as shown in §A.

RQ1 Answer: In all evaluations, the best scores from LLMs surpassed baselines, except for a 1% lag in syntax correctness. The highest scores were primarily achieved by GPT-4o, but open-source code models also performed well (*i.e.*, Deepseek-Coder and Codestral). Overall, despite struggles to execution correctness, LLMs have demonstrated their potential in JS deobfuscation, offering new insights for this problem.

5.3 RQ2: Syntax and Execution Correctness

To address RQ2, we closely examine results from the syntax and execution evaluators for both LLMs and our baseline. Figure 3 presents the detailed results, indicating the number of deobfuscated JS programs that successfully passed or failed the syntax and execution checks. First, we observe that the output generated by our baselines (Synchrony and JS-deobfuscator) passed all syntax checks, whereas the LLMs exhibited some failures. Specifically, Deepseek-Coder and GPT-4o achieved the lowest and highest syntax check failure rates, 0.94% and 4.01%, respectively, and the average syntax error rate is 2.76% across all LLMs, which is quite close to our baseline achievements. Notably, GPT-4o performed significantly worse than the other LLMs against the simple transformation of code-compact, and our manual inspection reveals that the safety filtering mechanism is triggered by the deobfuscation requests (more details can be found in §6.1), which caused the model to throw an exception to reject to response. However, the same problem did not arise in other requests.

On execution correctness, more significant gaps were observed among LLMs. Specifically, CodeLlama, Codestral, and Deepseek-Coder significantly outperform the two general-purpose models, namely Llama-3.1 and Mixtral, achieving an average 32.66% advantage across all transformations. Generally, besides the leading GPT-4o, LLMs still find it tricky to generate semantically correct deobfuscated JS code. Among all transformations, we notice that LLMs successfully deobfuscated the largest number of samples against the code compact transformation, achieving an average score of 0.7918 on execution correctness, and struggled the most with the string obfuscation transformation, which has an average score of 0.4663. In contrast, our baselines passed almost all execution checks, except in self-defending transformation, where only 2 out of 1,298 samples passed the checks. Our case studies also expose the predominant causes of errors in syntax and execution evaluations, which are detailed in §6.

As discussed in §4.1, developers often apply multiple obfuscation transformations to JS programs. To evaluate the deobfuscation performance of LLMs and our baselines in such scenarios, we analyze the average results across various combinations using a certain number of transformations, as presented in Figure 5. Overall, LLMs exhibit a noticeable decline in performance against increasingly complex obfuscation (*i.e.*, an increase in the number of transformations) in both syntax and execution evaluations. Also, both LLMs and the baselines almost completely fail in deobfuscation when the number of transformations exceeds six. For instance, as the number of transformations rises from one to seven, the syntax and execution correctness of GPT-4o decrease by 0.8077 and 0.8252, respectively. The GPT-4o has the best robustness against increasing obfuscated transformations, it maintains a 20.39% semantic checks pass rate even when the number of transformations increases to six, while other models drop to 2% on average. Meanwhile, the baselines maintained consistent syntax evaluation but struggled with execution correctness, with JS-deobfuscator and Synchrony experiencing score decreases of 0.8383 and 0.8344, respectively.

RQ2 Answer: Many LLM-deobfuscated JS programs struggle with syntax and execution evaluations, *e.g.*, failure rates of 2.76% and 37.40% on average, respectively, while the baseline methods achieved zero syntax errors and 16.25% semantics errors. The code compact and string obfuscation present the easiest and hardest challenges to LLMs. Both LLMs and baselines are sensitive to the number of obfuscation transformations applied. The combination of six obfuscation transformations suggests a threshold that severely hinders the deobfuscation efforts of all methods.

5.4 RQ3: Code Simplification and Similarity

To address RQ3, we evaluate LLMs and our baselines to simplify obfuscated JS programs, with results presented in Figure 4. Generally, LLMs achieve significantly better simplification scores across various transformations, compared to baseline methods. CodeLlama recorded the highest median simplification score of 0.7643 against string obfuscation, outperforming the scores of 0.6860 achieved by Synchrony. In contrast, JS-deobfuscator shows almost no complexity reduction with a score of 0.01. Facing obfuscation transformations like code compact and name obfuscation, which did not change the code structure too much causing less increasing in halstead length, both LLMs and baselines presented lower simplification scores than the scores achieved against the other transformations.

Furthermore, we explore how the combination of obfuscation transformations impacts the simplification performance of both LLMs and our baselines, and Figure 6 displays the average results across different combination samplings (detailed in §4.1). We observe that LLMs generally outperform our baseline methods, even when handling obfuscated JS programs with a greater number of transformations combined. For instance, Codestral achieved an average simplification score of 0.6320 across all combinations, whereas the score for Synchrony is 0.3703. Furthermore, the performance of LLMs shows an upward trend. For example, CodeLlama achieves a 0.5096 better simplification score when handling inputs with five transformations compared to just one. This improvement is primarily because multiple transformations dramatically increase the complexity of the obfuscated code (*i.e.*, $HLoC_{obf}$ in Equation 1), while LLMs consistently produce concise deobfuscated code (low $HLoC_{deobf}$), demonstrating their effectiveness in reducing complexity. It is worth noting that LLMs generate less syntactically correct code facing complex combinations (*i.e.*, up to six transformations), where the simplification scores are no longer representative, we thus plot these scores using dashed lines in Figure 6.

To assess how the deobfuscated results are similar to the source JS programs (*i.e.*, P and P'' defined in §2.1, indicating the readability of the deobfuscated code), we compute the CodeBLEU scores and present the results in Figure 7. In general, GPT-4o achieved the best similarity between the deobfuscated and original programs, with a median CodeBLEU score of 0.6616 across all transformations. While our baselines, Synchrony and JS-deobfuscator, respectively achieved median scores of 0.5944 and 0.4851, indicating less readability of the JS program obfuscated. The code compact is still the easiest transformation for LLMs in similarity evaluation, while the name obfuscation seemed to pose a bigger challenge, decreasing the score of GPT-4o to 0.5317.

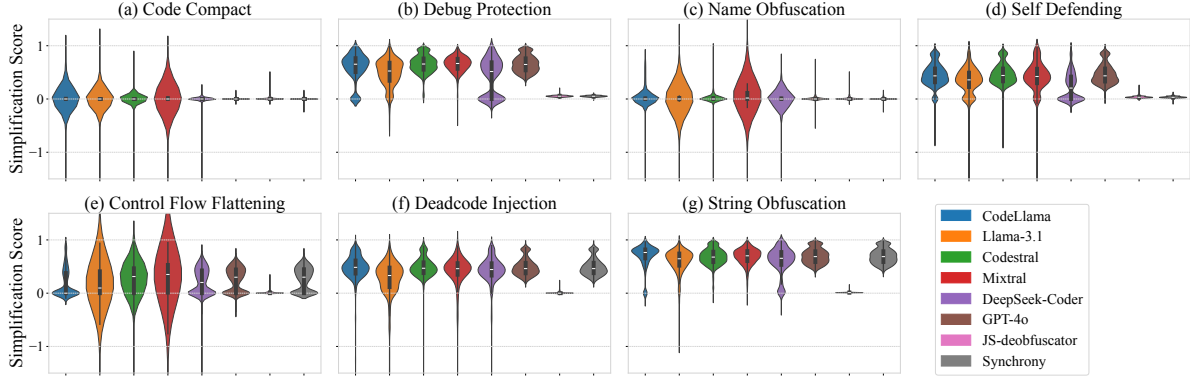


Figure 4: Code Simplification Scores of LLMs and Baselines across Obfuscation Transformations.

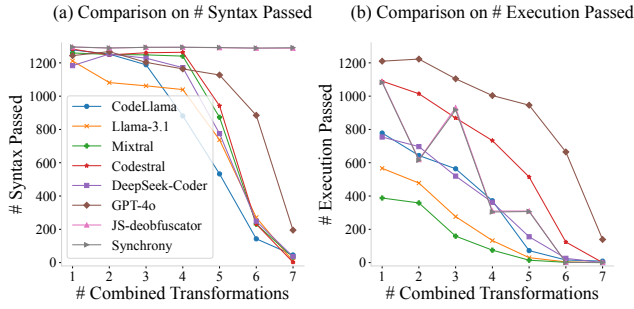


Figure 5: Syntax and Execution Correctness of LLMs and Baselines against Multiple Obfuscation Transformations.

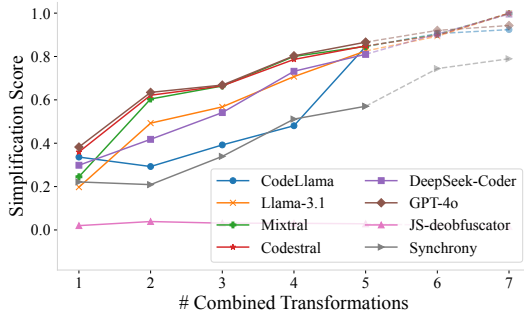


Figure 6: Code Simplification Scores of LLMs and Baselines against Multiple Obfuscation Transformations.

To further assess the impact of multiple obfuscation transformations, we present the code similarity scores in Figure 8. Similar to Figure 6, we have also dashed the lines where the number of transformations exceeds five. Generally, we observed a consistent decreasing trend in both LLMs and our baselines as the number of transformations increased. For instance, the CodeBLEU scores of Synchrony and Codestral decreases by 0.2154 and 0.1977, respectively, when transitioning from one transformation to five transformations. GPT-4o always maintained leadership as the complexity of obfuscation increased, and it exhibited a 0.5124 score

Table 5: Overall Deobfuscation Performance of LLMs and Baselines on Obfuscated JS Malware.

Model	Syntax Correctness	Execution Correctness	Simplification Scores	Similarity Scores	Time(s) Overhead
CodeLlama	0.9216	0.2228	0.1911	0.3611	5.61
Llama-3.1	0.5497	0.0904	0.3491	0.2611	8.78
Codestral	0.9280	0.2219	0.4998	0.2415	9.54
Mixtral	0.8631	0.0310	0.6363	0.1614	5.06
Deepseek-Coder	0.8932	0.2501	0.2931	0.3153	10.31
GPT-4o	0.3329	0.0773	0.5157	0.3294	N/A
JS-deobfuscator	1.0000	0.8414	0.0845	0.4048	0.31
Synchrony	1.0000	0.8620	0.1668	0.4423	0.21

against five combined transformations, outperforming other methods by an average of 21.62%. While Synchrony, the better among the baselines, achieved only 0.3787 CodeBLEU.

RQ3 Answer: Compared to baseline methods, LLMs generally demonstrate superior performance in simplifying obfuscated JS code and generating more human-readable code. Despite suffering from certain transformations, LLMs could achieve larger simplification scores, *e.g.*, CodeLlama recorded the highest median of 0.7643 in string obfuscation. Increasing the number of transformations makes it more difficult for LLMs to generate deobfuscation similar to original code, but allows LLMs to reduce code complexity even more.

5.5 RQ4: Deobfuscation on JS Malware

Since LLMs have demonstrated their ability to deobfuscate general JS programs, we further explore how they perform in handling obfuscated JS malware. As mentioned in §4.1, we have no test cases for JS malware to check the execution correctness. Therefore, we instead proxy by confirming the same behavior traces during malware execution, as stated in §4.3. Table 5 shows the overall results averaged across all transformations, and we also present more transformation-specific evaluation detailed in §A for further reference.

The code-specific models generally perform better in generating syntactically and semantically correct deobfuscation. Specifically,

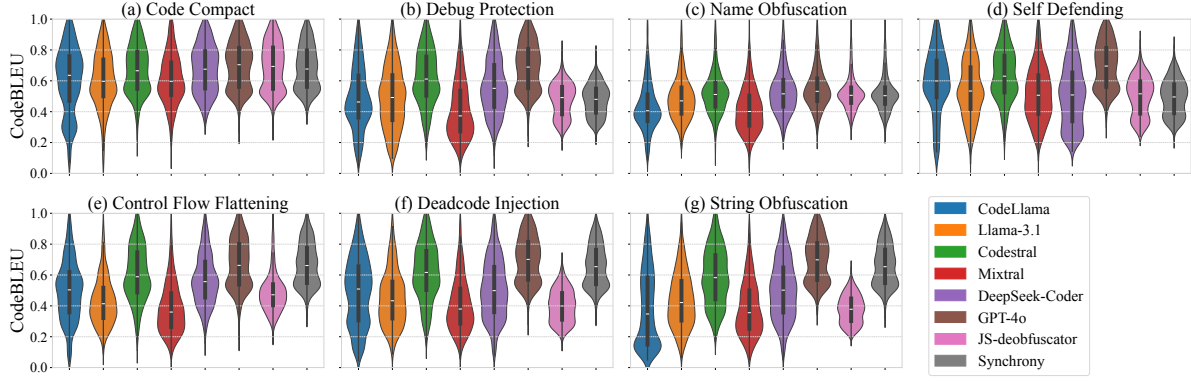


Figure 7: Code Similarity Scores of LLMs and Baselines across Obfuscation Transformations.

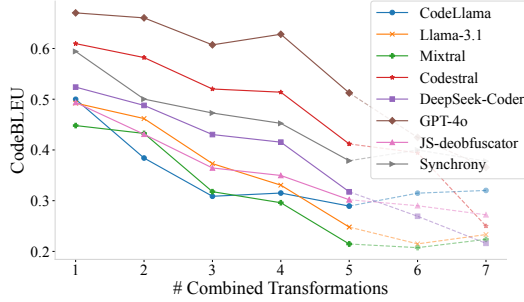


Figure 8: Code Similarity Scores of LLMs and Baselines against Multiple Obfuscation Transformations.

Codestral achieved the best syntax score of 92.80%, and Deepseek-Coder has the best execution correctness with a score of 25.01%. Meanwhile, we observe that the syntax and execution pass rates on malicious code are significantly lower than those on general code, dropping by an average of 22.43% (97.24% v.s. 74.81%) and 47.71% (62.60% v.s. 14.89%), respectively, indicating deobfuscating malware propose a significant challenge for LLMs. The requests of deobfuscating malicious code triggered the safety filtering and were rejected by GPT-4o, which resulted in its low syntax pass rate. In contrast, the baseline methods did not exhibit significant degradation in terms of syntactic and semantic correctness.

On the other hand, LLMs outperform baselines in simplifying obfuscated JS malware, 28.85% higher average score, indicating a more analyzable malicious code for human. Considering the poor performance in terms of execution correctness achieved by Mixtral and GPT-4o, we prefer to consider Codestral to have the best performance in simplification evaluation with a score of 49.98%. Finally, compared to the evaluation results on the general JS programs, deobfuscated malicious code has less similarity to the original malware, 26.72% lower average score. Our manual checking found that the effectively deobfuscated code are quite different from the original malware those had already been obfuscated, typically due to the inherent obfuscation. To help understand the reason, we present an example (detailed in Appendix §A) that has some string-obfuscation on the original JS malware.

RQ4 Answer: LLMs exhibit a potential in deobfuscating JS malware, and especially perform well in simplifying the obfuscated samples, outperforming baselines significantly. However, the models are still suffering from preserving code semantics when performing deobfuscation.

6 Case Studies

In this section, we study individual cases to get an in-depth analysis of LLM’s deobfuscation errors and better performance in code simplification and readability. We also provide some examples in Appendix §A for further reference.

6.1 LLM Deobfuscation Errors

In §5.3, we observe a small proportion of LLM-generated deobfuscation outputs that did not pass our syntax and execution checks. For this, we have performed a manual analysis on the failed cases and identified four categories of root causes as described below.

Self-repeating. Among the error samples, we often observe that the LLMs restate inputs or repeat outputs already produced up to the maximum context constraint. For instance, we have seen many cases where the natural language instructions provided in the prompt are included verbatim in the deobfuscation output, indicating an inability of the LLM to follow the instruction. Such outputs, when subjected to syntax checks, result in errors, indicating a failure in effective deobfuscation.

Limited LLM Context Window Size. Another category of errors occurs when the target deobfuscation output exceeds the context window size of LLMs. Specifically, there are cases in which LLMs task as input the long obfuscated JS program. These lengthy JS programs are generated by some transformations that significantly change the code structure. For example, the control flow flattening transformation converts the callees of a function into a hash map. The generation of the complete hash map surpasses the output limitation of LLMs. As a result, the generated deobfuscated code is incomplete, leading to syntax errors during our evaluations.

Refuse to Response. We mainly encountered rejected responses when performing the deobfuscation task with GPT-4o, especially when requesting to deobfuscating malicious JS programs. This

could be due to malicious code or deobfuscation requests triggering security filtering policies developed by OpenAI. Another possible reason is that the model's harmless alignment makes it believe that it should not respond to the requests.

Semantic Manipulation. Besides syntax errors, we also face execution errors due to semantic manipulations by LLMs, where they produce syntactically correct programs with altered semantics. For example, we have seen the case where the original JS program performs operations of concatenating input elements in the order of `line[2]`, `line[0]`, and `line[1]`. However, in the deobfuscation output, this order is erroneously altered to `line[1]`, `line[0]`, and `line[2]`, due to incorrectly recognizing `\x02` and `\x01` in the obfuscated code as 1 and 2, respectively.

6.2 Code Simplification and Readability

In §5.4, LLMs exhibit superior performance in simplifying obfuscated code and generating deobfuscated code more similar to the original JS programs compared to our baselines. To understand the underlying reasons for this, we analyzed cases where LLMs achieved median simplification and similarity scores. We observe that LLMs can successfully learn the obfuscated JS program semantics, recover function/variable names and types, and enhance the readability of JS programs. A detailed example is presented in Appendix §A for readers' interest. Such cases demonstrate that LLMs possess four key capabilities of JS deobfuscation.

(I) Precise Obfuscated JS Code Comprehension. The aforementioned example features a JS function designed to sum the digits of a user-provided input. Despite significant name obfuscation transformation that converts the program into a very noisy representation that overflows with alphanumeric jumbles and meaningless symbols, the Codestral model effectively produced not only clean but also *semantically correct* deobfuscated code, indicating the success of semantics comprehension.

(II) Accurate Function and Variable Name Recovery. In the obfuscated code of the above example, original variable names are replaced with nonsensical identifiers, e.g., `_0x5e1f19` and `_0x11dd03`, and the function names are obscured using a hash map. Nonetheless, Codestral successfully restores these names and even suggested improved variable names (`input` and `num`) in the deobfuscated code.

(III) Correct Type Inference. This example also shows that Codestral accurately infers the integer type for the variable `num`, which hosts the input number. And the model generates code preserving this type correctness (*i.e.*, using the `Math.floor` function to keep the integer type).

(IV) Significant Enhancement in Code Readability. While Codestral accurately predicts names and types, our baseline deobfuscator Synchrony generates code nearly identical to the obfuscated input, retaining all nonsensical variable names and greatly hindering program comprehension. Compared to Synchrony, Codestral's output is much easier to read and understand.

From this investigation, it is evident that LLMs significantly outperform our baselines in generating simplified and more human-readable deobfuscated JS code. This crucial finding is not always directly reflected in the statistical results, but is critical for real-world applications in security analysis.

7 Discussion

7.1 Lessons Learned

Potential of LLMs in JS Deobfuscation. In this paper, we have conducted a systematic study of state-of-the-art LLMs and existing deobfuscators (*i.e.*, our baselines) in the context of JS deobfuscation. While LLMs lag behind these baselines in syntax and execution evaluations (RQ2), they exhibit significant potential in generating simplified and easily understandable deobfuscated code (RQ3). Our case studies, detailed in §6.2, highlight four distinct advantages and properties of LLMs in deobfuscation, including a robust capacity for understanding obfuscated JS programs and effective name/type inference. These capabilities demonstrate that LLMs are not only valuable for deobfuscation tasks but could also be instrumental in addressing other research challenges within the field, such as improving type inference processes.

Enhancing LLMs for JS Deobfuscation. To enhance LLMs for deobfuscation tasks, we have pinpointed a crucial area for improvement: performance, specifically in terms of syntax and execution correctness (RQ2). Our additional case studies in §6.1 have identified key errors that need addressing, including self-repeating patterns, limitations related to LLM context window size, and errors in semantic manipulation. Although the context length of LLMs has been expanding recently, such as the Claude model supports up to 200k tokens (8k output tokens), further efforts are necessary to mitigate issues like self-repeating and semantic manipulation. Implementing advanced pretraining, fine-tuning, and other training methodologies could substantially enhance LLM capabilities in this domain. For instance, code-specific models, which have demonstrated superior performance than the general-purpose models in JsDEOBSBENCH evaluations (RQ1), benefits from being specifically fine-tuned on code datasets. Additionally, our investigation shows that complicated obfuscated code, such as more obfuscated transformations or the JS malware with more complex logic (RQ2, RQ3, and RQ4), downgrade the effectiveness and readability of LLMs' deobfuscation. We hypothesize that increasing the exposure to such examples, possibly through extended fine-tuning and continuous pretraining, could further augment LLMs' deobfuscation performance. This approach suggests a promising direction for future research to enhance LLMs in complex deobfuscation tasks.

7.2 Limitations

Dataset. In this paper, we construct a new, large-scale, and execution-verifiable obfuscated JS dataset from scratch. For dataset construction, we selected challenge-solving programs as our data source, adhering to the common practice in LLM benchmarks [5, 24]. However, the JS community is rapidly evolving, and the volume and diversity of JS programs are increasing significantly. Thus, including a broader array of JS programs, e.g., recently developed or malware scripts, represents an intriguing direction for future research. For the malware dataset in §4.1, we observe the mixed sets of obfuscated and clean JS programs without obfuscation. In our best efforts, we are unable to find open-source clean malware source program datasets or feasible ways to filter out clean malware programs without obfuscation in a large scale. This mixed malware JS dataset causes relatively low similarity scores as presented in §5.5. To this

end, we believe the evaluations upon clean malware JS datasets without any obfuscation can be an interesting future work.

Obfuscation and Deobfuscation Approaches. When developing JsDeObsBENCH, we apply common transformations using the prevalent obfuscator, JS-Obfuscator, to JS programs. We note that the landscape of obfuscators is expanding, with new transformations being proposed, such as opportunistic transformation-based obfuscation [64]. On the deobfuscation front, new tools are emerging that could potentially serve as baselines for future evaluations. That said, exploring these obfuscation and deobfuscation techniques and assessing their effectiveness with the JsDeObsBENCH benchmark could constitute an intriguing avenue for future research.

Advanced LLMs. Currently, we focus on code-specific large language models (LLMs) that have demonstrated state-of-the-art capabilities in program understanding and have ranked highly in benchmarks. However, we have not exhaustively explored all potentially suitable LLMs, nor have we examined models of varying sizes beyond those specifically focused on in our study. Additionally, we recognize that the research and development of LLMs is a rapidly evolving field, attracting significant attention from both academia and industry. This has led to the introduction of an increasing number of new LLMs that could further enhance code LLM capabilities. Given that our work is conducted by an academic group with limited computational resources, it remains challenging to conduct exhaustive studies on all available models.

8 Conclusion

We have presented JsDeObsBENCH, a framework for systematically assessing the capabilities of LLMs for JS deobfuscation. Our comprehensive evaluation reveals that while models like GPT-4o significantly enhance code simplification and readability, they also expose inherent challenges in maintaining syntax accuracy and execution reliability. These findings highlight the potential of LLMs to transform web security practices by automating the deobfuscation process, yet also highlight the necessity for ongoing improvements in model training and optimization. As LLMs continue to evolve, we believe JsDeObsBENCH can serve as an invaluable tool for benchmarking advancements and guiding future developments in the field.

Acknowledgments

We thank Xiuwei Shang as well as the anonymous reviewers for their insightful feedback and valuable discussions. This work was supported in part by NSF awards 2112471 and 2330264, and the Schmidt Sciences Safety Science award. Any opinions, findings, and conclusions expressed in this paper are the authors' and do not reflect those of the sponsors.

References

- [1] 2024. Collection of almost 40,000 javascript malware samples. <https://github.com/HynekPetrak/javascript-malware-collection>.
- [2] 2024. DeepSpeed. <https://github.com/microsoft/DeepSpeed>.
- [3] 2024. A deobfuscator for scripts obfuscated by Obfuscator.io. <https://github.com/ben-sb/obfuscator-io-deobfuscator>.
- [4] 2024. Docker Official Image, Node.js is a JavaScript-based platform for server-side and networking applications. https://hub.docker.com/_/node.
- [5] 2024. EvalPlus Leaderboard. <https://evalplus.github.io/leaderboard.html>.
- [6] 2024. General purpose JavaScript deobfuscator. <https://github.com/ben-sb/javascript-deobfuscator>.
- [7] 2024. Hugging Face Hub. <https://huggingface.co/docs/hub/index>.
- [8] 2024. A JavaScript checker and optimizer. <https://github.com/google/closure-compiler>.
- [9] 2024. javascript cleaner & deobfuscator. <https://github.com/relative/synchrony>.
- [10] 2024. JavaScript Deobfuscator and Unpacker. <https://github.com/lelinhtinh/de4js>.
- [11] 2024. JavaScript Minifier is an easy-to-use tool for minifying JavaScript code. <https://www.toptal.com/developers/javascript-minifier>.
- [12] 2024. JavaScript parser / mangler / compressor / beautifier toolkit. <https://github.com/mishoo/UglifyJS>.
- [13] 2024. JS reverse minifier based on statistical machine translation. <https://github.com/bvasiles/jsNaughty>.
- [14] 2024. Node.js® is a free, open-source, cross-platform JavaScript runtime environment that lets developers create servers, web apps, command line tools and scripts. <https://nodejs.org/>.
- [15] 2024. Obfuscate JavaScript (beyond repair) with Ruby. <https://github.com/rapid7/jsobfu>.
- [16] 2024. Obfuscate string literals in JavaScript code. <https://github.com/anseki/gnirts>.
- [17] 2024. A powerful obfuscator for JavaScript and Node.js. <https://github.com/javascript-obfuscator/javascript-obfuscator>.
- [18] 2024. PyTorch. <https://pytorch.org/>.
- [19] 2024. STATISTICAL RENAMING, TYPE INFERENCE AND DEOBFUSCATION. <http://jsnice.org/>.
- [20] 2024. This repository contains a list of pseudo-sorted malicious JavaScripts collected from time to time. <https://github.com/geeksonsecurity/js-malicious-dataset>.
- [21] 2024. Transformers. <https://huggingface.co/>.
- [22] 2024. A utility to analyze malicious JavaScript. <https://github.com/CapacitorSet/box-js>.
- [23] Rohan Bavishi, Michael Pradel, and Koushik Sen. 2018. Context2Name: A Deep Learning-Based Approach to Infer Natural Variable Names from Usage Contexts. doi:10.48550/arXiv.1809.05193
- [24] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [25] Nicholas Carlini, Florian Tramer, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Ulfr Erlingsson, et al. 2021. Extracting training data from large language models. In *30th USENIX Security Symposium (USENIX Security 21)*. 2633–2650.
- [26] Guoqiang Chen, Xin Jin, and Zhiqiang Lin. 2025. JsDeObsBench: Measuring and Benchmarking LLMs for JavaScript Deobfuscation. arXiv:2506.20170 [cs.CR] <https://arxiv.org/abs/2506.20170>
- [27] Guoqiang Chen, Huiqi Sun, Daguang Liu, Zhiqi Wang, Qiang Wang, Bin Yin, Lu Liu, and Lingyun Ying. 2025. ReCopilot: Reverse Engineering Copilot in Binary Analysis. arXiv:2505.16366 [cs.CR] <https://arxiv.org/abs/2505.16366>
- [28] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [29] Charlie Curtis, Benjamin Livshits, Benjamin Zorn, and Christian Seifert. 2011. ZOZZLE: Fast and precise In-Browser JavaScript malware detection. In *20th USENIX Security Symposium (USENIX Security 11)*.
- [30] Antcode developers. [n. d.]. Atcoder. <https://atcoder.jp/> Accessed on 11-29-2024.
- [31] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).
- [32] Yangruibo Ding, Zijian Wang, Wasi Ahmad, Hantian Ding, Ming Tan, Nihal Jain, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, et al. 2024. Crosscodeeval: A diverse and multilingual benchmark for cross-file code completion. *Advances in Neural Information Processing Systems* 36 (2024).
- [33] ECMA International. 2017. ECMA-262: ECMAScript Language Specification. <https://ecma-international.org/publications-and-standards/standards/ecma-262/>.
- [34] Aurore Fass, Michael Backes, and Ben Stock. 2019. Hidenoseek: Camouflaging malicious javascript in benign asts. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. 1899–1913.
- [35] Aurore Fass, Robert P Krawczyk, Michael Backes, and Ben Stock. 2018. Jast: Fully syntactic detection of malicious (obfuscated) javascript. In *Detection of Intrusions and Malware, and Vulnerability Assessment: 15th International Conference, DIMVA 2018, Saclay, France, June 28–29, 2018, Proceedings 15*. Springer, 303–325.
- [36] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence. arXiv:2401.14196 [cs.SE]
- [37] Maurice H Halstead. 1977. *Elements of Software Science (Operating and programming systems series)*. Elsevier Science Inc.

- [38] Adrian Herrera. 2020. Optimizing Away JavaScript Obfuscation. In *Source Code Analysis and Manipulation 2020 (SCAM'20)*. arXiv. doi:10.48550/arXiv.2009.09170 arXiv:2009.09170 [cs].
- [39] Ariya Hidayat. 2024. ECMAScript parsing infrastructure for multipurpose analysis. <https://esprima.org>.
- [40] Peiwei Hu, Ruigang Liang, and Kai Chen. 2024. DeGPT: Optimizing Decompiler Output with LLM. In *Proceedings 2024 Network and Distributed System Security Symposium (2024)*. <https://api.semanticscholar.org/CorpusID.267622140>.
- [41] Grégoire Jacob, Hervé Debar, and Eric Filiol. 2008. Behavioral detection of malware: from a survey towards an established taxonomy. *Journal in computer Virology* 4 (2008), 251–266.
- [42] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Léo Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2024. Mixtral of Experts. arXiv:2401.04088 [cs.LG]
- [43] Linxi Jiang, Xin Jin, and Zhiqiang Lin. 2025. Beyond Classification: Inferring Function Names in Stripped Binaries via Domain Adapted LLMs. In *Proceedings 2025 Network and Distributed System Security Symposium*. San Diego, CA, USA.
- [44] Xin Jin, Jonathan Larson, Weiwei Yang, and Zhiqiang Lin. 2023. Binary code summarization: Benchmarking chatgpt/gpt-4 and other large language models. arXiv preprint arXiv:2312.09601 (2023).
- [45] Xin Jin, Kexin Pei, Jun Yeon Won, and Zhiqiang Lin. 2022. Symlm: Predicting function names in stripped binaries via context-sensitive execution-aware code embeddings. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 1631–1645.
- [46] Mehran Jodavi, Mahdi Abadi, and Elham Parhizkar. 2015. JSObfusDetector: A binary PSO-based one-class classifier ensemble to detect obfuscated JavaScript code. In *2015 The International Symposium on Artificial Intelligence and Signal Processing (AISIP)*. IEEE, 322–327.
- [47] Song Li, Mingqing Kang, Jianwei Hou, and Yinzhi Cao. 2022. Mining node.js vulnerabilities via object dependence graph and query. In *31st USENIX Security Symposium (USENIX Security 22)*. 143–160.
- [48] Yi Li, Aashish Yadavally, Jiaxing Zhang, Shaohua Wang, and Tien N Nguyen. 2023. DeMinify: Neural Variable Name Recovery and Type Inference. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 758–770.
- [49] Han Liu, Chengnian Sun, Zhendong Su, Yu Jiang, Ming Gu, and Jianguang Sun. 2017. Stochastic optimization of program obfuscation. In *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. IEEE, 221–231.
- [50] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. 2023. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *Comput. Surveys* 55, 9 (2023), 1–35.
- [51] Meta AI. 2025. Introducing Llama 3.1: Our most capable models to date. <https://ai.meta.com/blog/meta-llama-3-1/>. Accessed: 2025-01-01.
- [52] Mistral AI. 2025. Codestral: Hello, World! Empowering developers and democratising coding with Mistral AI. <https://mistral.ai/news/codestral/>. Accessed: 2025-01-01.
- [53] Marvin Moog, Markus Demmel, Michael Backes, and Aurore Fass. 2021. Statically Detecting JavaScript Obfuscation and Minification Techniques in the Wild. In *2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN) (DSN 2021)*. 569–580. doi:10.1109/DSN48987.2021.00065 ISSN: 2158-3927.
- [54] OpenAI. 2025. Hello GPT-4 Turbo. <https://openai.com/index/hello-gpt-4o/>. Accessed: 2025-01-08.
- [55] OpenAI. 2025. OpenAI API. <https://openai.com/api/>. Accessed: 2025-01-07.
- [56] Rangeet Pan, Ali Reza Ibrahimzade, Rahul Krishna, Divya Sankar, Lambert Pouguem Wassi, Michele Merler, Boris Sobolev, Raju Pavuluri, Saurabh Sinha, and Reyhaneh Jabbarvand. 2023. Understanding the effectiveness of large language models in code translation. arXiv preprint arXiv:2308.03109 (2023).
- [57] Ruchir Puri, David Kung, Geert Janssen, Wei Zhang, Giacomo Domeniconi, Vladimir Zolotov, Julian Dolby, Jie Chen, Mihir Choudhury, Lindsey Decker, Veronika Thost, Luca Buratti, Saurabh Pujar, Shyam Ramji, Ulrich Finkler, Susan Malaika, and Frederick Reiss. 2021. CodeNet: A Large-Scale AI for Code Dataset for Learning a Diversity of Coding Tasks.
- [58] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. 2018. Improving language understanding by generative pre-training. (2018).
- [59] Sampsa Rauti and Ville Leppänen. 2018. A Comparison of Online JavaScript Obfuscators. In *2018 International Conference on Software Security and Assurance (ICSSA) (ICSSA 2018)*. 7–12. doi:10.1109/ICSSA45270.2018.00012
- [60] Veselin Raychev, Martin Vechev, and Andreas Krause. 2015. Predicting program properties from "big code". *ACM SIGPLAN Notices* 50, 1 (2015), 111–124.
- [61] Kunlun Ren, Weizhong Qiang, Yueming Wu, Yi Zhou, Deqing Zou, and Hai Jin. 2023. An Empirical Study on the Effects of Obfuscation on Static Machine Learning-Based Malicious JavaScript Detectors. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1420–1432.
- [62] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, M. Zhou, Ambrosio Blanco, and Shuai Ma. 2020. CodeBLEU: a Method for Automatic Evaluation of Code Synthesis. arXiv abs/2009.10297 (2020). <https://arxiv.org/abs/2009.10297>
- [63] Alan Romano, Daniel Lehmann, Michael Pradel, and Weihang Wang. 2022. Wobfuscator: Obfuscating javascript malware via opportunistic translation to webassembly. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1574–1589.
- [64] Alan Romano, Daniel Lehmann, Michael Pradel, and Weihang Wang. 2022. Wobfuscator: Obfuscating JavaScript Malware via Opportunistic Translation to WebAssembly. In *2022 IEEE Symposium on Security and Privacy (SP) (SP 2022)*. 1574–1589. doi:10.1109/SP46214.2022.9833626 ISSN: 2375-1207.
- [65] Alan Romano, Yunhui Zheng, and Weihang Wang. 2020. Minerray: Semantics-aware analysis for ever-evolving cryptojacking detection. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*. 1129–1140.
- [66] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2024. Code Llama: Open Foundation Models for Code. arXiv:2308.12950 [cs.CL]
- [67] Andrea Saracino, Daniele Sgandurra, Gianluca Dini, and Fabio Martinelli. 2016. Madam: Effective and efficient behavior-based android malware detection and prevention. *IEEE Transactions on Dependable and Secure Computing* 15, 1 (2016), 83–97.
- [68] Shaown Sarker, Jordan Jueckstock, and Alexandros Kapravelos. 2020. Hiding in plain site: Detecting javascript obfuscation through concealed browser api usage. In *Proceedings of the ACM Internet Measurement Conference*. 648–661.
- [69] Xiuwei Shang, Guoqiang Chen, Shaoyin Cheng, Shikai Guo, Yanming Zhang, Weiming Zhang, and Nenghai Yu. 2025. FoC: Figure out the Cryptographic Functions in Stripped Binaries with LLMs. *ACM Trans. Softw. Eng. Methodol.* (April 2025). doi:10.1145/3731449
- [70] Xiuwei Shang, Guoqiang Chen, Shaoyin Cheng, Benlong Wu, Li Hu, Gangyang Li, Weiming Zhang, and Nenghai Yu. 2025. BinMetric: A Comprehensive Binary Analysis Benchmark for Large Language Models. arXiv:2505.07360 [cs.SE] <https://arxiv.org/abs/2505.07360>
- [71] Mikhail Shcherbakov, Musard Balliu, and Cristian-Alexandru Staiacu. 2023. Silent spring: Prototype pollution leads to remote code execution in Node.js. In *32nd USENIX Security Symposium (USENIX Security 23)*. 5521–5538.
- [72] Philippe Skolka, Cristian-Alexandru Staiacu, and Michael Pradel. 2019. Anything to hide? studying minified and obfuscated code in the web. In *The world wide web conference*. 1735–1746.
- [73] Kwangwon Sun and Sukyoung Ryu. 2017. Analysis of JavaScript programs: Challenges and research trends. *ACM Computing Surveys (CSUR)* 50, 4 (2017), 1–34.
- [74] Hieu Tran, Ngoc Tran, Son Nguyen, Hoan Nguyen, and Tien N Nguyen. 2019. Recovering variable names for minified code with usage contexts. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 1165–1175.
- [75] Alessio Trivisonno. [n. d.]. Reverse engineering : Code Deobfuscation in the age of AI. <https://infosecwriteups.com/the-cybersecurity-revolution-at-the-age-of-ai-openai-and-code-deobfuscation-3f9dd278b900>.
- [76] Bogdan Vasilescu, Casey Casalnuovo, and Premkumar Devanbu. 2017. Recovering clear, natural identifiers from obfuscated JS names. In *Proceedings of the 2017 11th international meeting on foundations of software engineering*. 683–693.
- [77] Yutaka Watanobe. [n. d.]. Aizu online judge. <https://onlinejudge.u-aizu.ac.jp>
- [78] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. 2022. Emergent abilities of large language models. arXiv preprint arXiv:2206.07682 (2022).
- [79] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [80] Yuqing Yang, Yue Zhang, and Zhiqiang Lin. 2025. Understanding Miniapp Malware: Identification, Dissection, and Characterization. In *Proceedings 2025 Network and Distributed System Security Symposium*. San Diego, CA, USA.
- [81] Yue Zhang, Bayan Turkistani, Allen Yuqing Yang, Chaoshun Zuo, and Zhiqiang Lin. 2021. A Measurement Study of Wechat Mini-Apps. *Proc. ACM Meas. Anal. Comput. Syst.* 5, 2, Article 14 (June 2021), 25 pages. doi:10.1145/3460081
- [82] Zibin Zheng, Kaiwen Ning, Yanlin Zhang, Jingwen Zhang, Dewu Zheng, Mingxi Ye, and Jiachi Chen. 2023. A survey of large language models for code: Evolution, benchmarking, and future trends. arXiv preprint arXiv:2311.10372 (2023).
- [83] Joe Zimmerman. 2015. How to obfuscate programs directly. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 439–467.

Imagine you are a skilled JavaScript developer, skilled in code obfuscation and reverse engineering. I will provide you with an obfuscated JavaScript code, and your task is to output the deobfuscated code, wrapped in three backticks (```)

Input obfuscated JavaScript code:

```
```javascript
{obfuscated_code}
```
```

Output deobfuscated JavaScript code:

(a) Zero-shot Prompt

Imagine you are a skilled JavaScript developer, skilled in code obfuscation and reverse engineering. I will provide you with an obfuscated JavaScript code, and your task is to output the deobfuscated code, wrapped in three backticks (```)

Example obfuscated JavaScript code:

```
```javascript
{example_obfuscated_code}
```
```

Example deobfuscated JavaScript code:

```
```javascript
{example_original_code}
```
```

Input obfuscated JavaScript code:

```
```javascript
{obfuscated_code}
```
```

Output deobfuscated JavaScript code:

(b) One-shot Prompt

Figure 9: In-context Learning Prompts for LLM-based JS De-obfuscation.

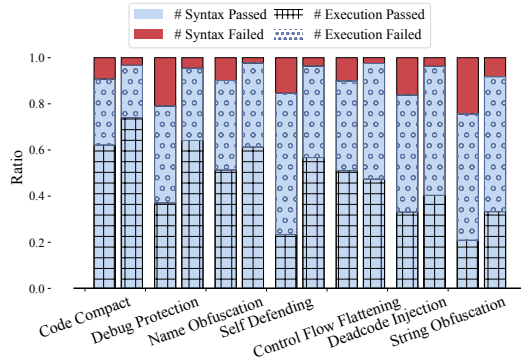


Figure 10: Syntax and Execution Correctness of LLMs with Zero-shot (Left Bar) and One-shot (Right Bar) Prompts across Obfuscation Transformations.

A Appendix

Due to space constraints, the full appendix of this paper, which includes detailed evaluation scores, prompt designs, and several case studies for interested readers, is provided in the extended version of this paper at [26].

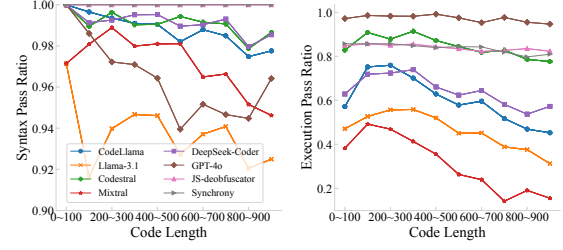


Figure 11: Syntax and Execution Correctness Evaluation across Different Code Lengths (at the Character Level).

A.1 In-context Learning Prompt Design

Considering that it is difficult for LLMs to get sufficient JavaScript deobfuscation training, even for the code-specific LLMs, we have utilized in-context learning to instruct the model to perform deobfuscation. As shown in Figure 9, we have developed two approaches: zero-shot and one-shot prompting. The zero-shot prompt (Figure 9a) contains the specified role, a description of the task, and a prescribed output format. The one-shot prompt (Figure 9b) enriches this setup by including demonstration examples of both obfuscated and correctly deobfuscated code.

To evaluate the effectiveness of these two prompting approaches, we conducted experiments across various obfuscation transformations. As shown in Figure 10, the demonstration examples in one-shot prompts significantly enhance LLMs' deobfuscation performance. Specifically, compared to zero-shot prompting, using one-shot prompts improves the syntax and execution correctness by 11.09% and 14.03%, respectively. These improvements demonstrate that providing contextual examples helps LLMs better understand the deobfuscation task and generate more accurate results. The success of one-shot prompting suggests that exposing LLMs to example pairs of obfuscated and deobfuscated code is an effective strategy for improving their deobfuscation capabilities.

A.2 Code Length Impact on Deobfuscation

We further examine how code length impacts the effectiveness of LLMs. Specifically, we focus on presenting the syntax and execution correctness, areas where LLMs showed poorer performance in our prior assessments. Figure 11 presents the syntax and execution correctness as a ratio of the number of samples that passed the checks. The results indicate that the general models show a noticeable decreasing performance as code length increases. For instance, the syntax and execution correctness for the Mixtral, drop by 0.0345 and 0.3369 respectively, as the code length range increases from (100, 200) to (900, 1000). While code-LLMs like Codestral exhibit better robustness with only 0.0097 and 0.0521 degradations. This trend is attributed to the autoregressive nature of LLMs, where each token of the deobfuscation output is generated based on previously generated tokens. Therefore, any errors in earlier tokens can propagate and affect the generation of subsequent tokens, compounding inaccuracies as the sequence lengths.